

Dropbox

File sync – Dropbox · Google Drive · OneDrive. Server వైపు ఇది object storage, ఒక పరిష్కరించిన problem.

కష్టమంతా client వైపు ఉంది: ఒక పరికరం, ఒక నెమ్మదైన network, మరియు ఒక మారుతున్న వృక్షం.

మూడు విరుపులు. మొదటిది – **ఒక byte** మారితే 16 MB upload, మరియు స్థిరమైన ముక్కలు దాన్ని **పరిష్కరించవు**. రెండోది – రెండు పరికరాలు రెండు **పూర్తిగా చట్టబద్ధమైన** తరలింపులు చేస్తే వృక్షం తెగిపోయి **221 to 213 files** అదృశ్యమవుతాయి. మూడోది – 10 లక్షల చిన్న files కి **13.9 గంటలు** పడుతుంది, అందులో bytes కి పట్టింది **5.5 నిమిషాలు**.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` `lo` run చేసినదే.

ఈ **PROBLEM LO** ఒక ఉచ్చు ఉంది, మరియు అది మొదటి 5 నిమిషాల్లోనే పడుతుంది

"Dropbox design చెయ్యి" అంటే చాలామంది **server** గురించి మాట్లాడతారు – S3, metadata database, CDN, sharding.

కానీ ఆ భాగం ఈ series lo ఇప్పటికే మూడుసార్లు వచ్చింది. Object storage ఒక పరిష్కరించిన problem.

ఈ problem యొక్క నిజమైన విషయం వేరే: ఒక వాడుకరి laptop, 100 Mbps, అప్పుడప్పుడూ offline, మరియు ఒక మారుతున్న **file system**. మీరు పంపాల్సింది data కాదు – **తేడా**. మరియు "తేడా" ని లెక్కించడం మీరు అనుకున్నదాని కంటే చాలా కష్టం.

కాబట్టి ఈ doc మొత్తం **client** వైపు ఉంది.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇక్కడ రెండు problems ఎందుకు ఉన్నాయి
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: ఏమి పంపాలి

3. Step – file మారితే దాన్ని upload చెయ్యి
4. మొదటి విరుపు – ఒక byte కి 16 MB
5. Step – స్థిరమైన ముక్కలు · మరియు అవి ఎప్పుడు పనిచేస్తాయి
6. అదే విరుపు, మళ్ళీ – మొదట్లో ఒక byte చేర్చితే
7. Step – విషయాధారిత విభజన (content-defined chunking)

Part 3 – రెండో విరుపు: వృక్షం

8. Step – file system ఒక వృక్షం, ఒక జాబితా కాదు
9. రెండో విరుపు – రెండు చట్టబద్ధమైన తరలింపులు, 213 files అదృశ్యం
10. Step – స్థితి కాదు, ఆపరేషన్లు · మొత్తం క్రమం · వలయ తిరస్కరణ

Part 4 – మూడో విరుపు: ఎన్నిసార్లు అడగాలి

11. మూడో విరుపు – 13.9 గంటలు నిరీక్షణ, 5.5 నిమిషాలు పని
12. Step – ముద్ద చేయడం · మరియు సమతుల్య file పరిమాణం

Part 5 – నిరూపణ మరియు ప్రదర్శన

13. మొత్తం code · 340 MB · mutation testing
14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటీస్ చెప్పాల్సిన English script
16. Follow-ups – dedup, cache, LAN sync, పెద్ద folders
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇక్కడ రెండు problems ఎందుకు ఉన్నాయి

ఒక folder. చాలా పరికరాలు. ఒక చోట ఏమి మారినా, మిగతా చోట్ల **వీలైనంత త్వరగా** అదే ఉండాలి. పరికరాలు offline కి వెళ్ళొచ్చు, మరియు తిరిగి వచ్చాక కలపాలి.

మొదట ఒక ప్రశ్న: **ఒక సాధారణ folder lo files ఎలా ఉంటాయి?**

1 మిలియన్ files · మొత్తం 165.96 GB

శతమానం | file పరిమాణం

p50		9.7 KB
p90		208.7 KB
p99		2.5 MB
p99.9		15.9 MB
p99.99		76.5 MB
సగటు		174.0 KB

(ఇది ఒక *log-normal* నమూనా, కొలిచిన data కాదు – కానీ ఏ file system ని చూసినా ఈ ఆకారమే: సగటు median కంటే **18 రెట్లు** పెద్దది.)

files lo ఈ భాగం | మొత్తం bytes lo వాటి వాటా | operations lo వాటి వాటా

చిన్న 90%		13.0%		90%
తర్వాతి 9%		33.6%		9%
పెద్ద 1%		53.4%		1%

ఈ పట్టిక చెబుతోంది: ఇది ఒక **PROBLEM** కాదు, రెండు

పెద్ద 1% files మీ bytes lo సగానికి పైగా ఉన్నాయి. వాటికి కావాల్సింది – **మారిన భాగాన్ని మాత్రమే పంపడం**. అది §7.

చిన్న 90% files మీ bytes lo 13% మాత్రమే, కానీ మీ requests lo **90%**. వాటికి కావాల్సింది – **తక్కువసార్లు అడగడం**. అది §12.

ఈ రెండు optimisations **ఒకదానికొకటి సంబంధం లేనివి**, మరియు ఒకటి చేసి రెండేది వదిలేస్తే system ఇప్పటికీ నెమ్మదిగానే ఉంటుంది.

ఒక్క సగటు సంఖ్య ("సగటు file 174 KB") ఈ రెండింటినీ దాచేస్తుంది.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఎన్ని files, ఏ పరిమాణాల్లో?	§1. ఒక్క సగటు సంఖ్య రెండు వేర్వేరు problems ని దాచేస్తుంది
Files ఎలా మారతాయి – చివర్లో కలపడమా, మధ్యలో సవరించడమా?	§4-§7. Log file మరియు Word document పూర్తిగా వేరు
Offline ఎంతసేపు ఉండొచ్చు?	§9 – ఎక్కువసేపైతే conflicts ఎక్కువ
ఒక folder ని తరలిస్తే ఏమవ్వాలి?	§9. ఇది delete + create కాదు, మరియు ఆ తేడా ముఖ్యం
ఇద్దరు ఒకే file ని మారిస్తే?	సాధారణంగా "conflicted copy" – కానీ directory కి అది పనిచేయదు
Round-trip ఎంత?	§11. ఇదే చిన్న files కి మొత్తం లెక్క
వాడుకరులు ఒకే data ని పంచుకుంటారా?	అవును అంటే cross-user dedup – §16
ఏమి మారిందో ఎలా తెలుస్తుంది?	File system watcher, లేక పూర్తి scan – §16

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా రాశారని తెలుస్తుంది

"ఒక 2 GB file యొక్క మొదట్లో ఒక byte చేర్చితే ఎన్ని bytes పంపాలి?"

ఎందుకంటే సహజమైన జవాబు – "ముక్కలుగా విడగొట్టి మారిన ముక్కని పంపుతాను" – **ఈ ఒక్క సందర్భంలో పూర్తిగా విఫలమవుతుంది**, మరియు §6 lo కొలుస్తాం: స్థిరమైన ముక్కలతో జవాబు **మొత్తం 2 GB**.

ఈ ప్రశ్న అడగడం అంటే మీకు తేడా తెలుసు: **ముక్కలు చేయడం ఒక విషయం; ముక్కల *సరిహద్దులు* ఎక్కడ పడతాయో అనేది వేరే విషయం.**

Part 2 – మొదటి విరుపు: ఏమి పంపాలి

3. Step – file మారితే దాన్ని upload చెయ్యి

అత్యంత సహజమైన sync:

```
watch(folder, (file) => {  
  if (hash(file) !== lastKnown[file]) upload(file);    // మారింది → పంపు  
});
```

ఒక hash పోల్చడం, మారితే పంపడం. **ఇది సరైన మొదలు**, మరియు చిన్న files కి ఇది ఖచ్చితంగా సరైనది.

ఇప్పుడు ఒక 16 MB file తీసుకుందాం – ఒక video project, ఒక database dump, ఒక VM image.

4. మొదటి విరుపు – ఒక byte కి 16 MB

16 MB file · ఒక్కో సవరణ తర్వాత ఎన్ని bytes మళ్ళీ upload కావాలి?

సవరణ	మొత్తం file	స్థిరమైన 4 MB
చివర్లో 1 MB కలపడం	17.00 MB (100%)	1.00 MB (6%)
మధ్యలో 1 MB మార్పడం	16.00 MB (100%)	4.00 MB (25%)
1 byte మార్పడం	16.00 MB (100%)	4.00 MB (25%)

ఒక్క BYTE మారితే 16 MB – మరియు ఇది సాధారణం కంటే ఎక్కువ జరుగుతుంది

ఒక Word document ని save చేస్తే metadata మారుతుంది. ఒక log కి ఒక పంక్తి కలిపితే file మారుతుంది. ఒక database file ని తాకితే ఒక page మారుతుంది.

100 Mbps మీద 16 MB అంటే **1.3 సెకన్లు** – ప్రతి save కి. Auto-save ప్రతి 30 సెకన్లకే ఉంటే, మీరు ఆ ఒక్క file కోసం మీ upload bandwidth లో **4%** శాశ్వతంగా ఖర్చు పెడుతున్నారు.

5. Step – స్థిరమైన ముక్కలు · మరియు అవి ఎప్పుడు పనిచేస్తాయి

స్పష్టమైన పరిష్కారం: file ని ముక్కలుగా విడగొట్టి, **మారిన ముక్కలని మాత్రమే** పంపడం.

```
for (let i = 0; i < buf.length; i += SIZE) chunks.push(buf.subarray(i, i+SIZE));
```

ముక్క చిన్నదైతే ఇంకా మెరుగు:

సవరణ	స్థిరమైన 4 MB	స్థిరమైన 64 KB
చివర్లో 1 MB కలపడం	1.00 MB (6%)	1.00 MB (6%)
మధ్యలో 1 MB మార్పడం	4.00 MB (25%)	1.00 MB (6%)
1 byte మార్పడం	4.00 MB (25%)	0.06 MB (0%)
మొదట్లో 1 MB తొలగించడం	15.00 MB (100%)	0.00 MB (0%)

64 KB ముక్కలతో ఒక byte మార్పు **64 KB** అవుతుంది. మరియు చివరి వరుస అద్భుతంగా ఉంది – మొదట్లో 1 MB తొలగిస్తే **సున్నా bytes**.

అది ఒక ఉచ్చు. ఎందుకో చూద్దాం.

6. అదే విరుపు, మళ్ళీ – మొదట్లో ఒక byte చేర్చితే

సవరణ	స్థిరమైన 4 MB	స్థిరమైన 64 KB	విషయాధారిత ~64 KB
మొదట్లో 1 MB తొలగించడం	15.00 MB (100%)	0.00 MB (0%)	0.02 MB (0%)
మొదట్లో 1 MB + 1 తొలగించడం	15.00 MB (100%)	15.00 MB (100%)	0.02 MB (0%)
మొదట్లో 1 byte చేర్చడం	16.00 MB (100%)	16.00 MB (100%)	0.08 MB (1%)

పై రెండు వరుసల మధ్య తేడా ఒక్క **BYTE** • ఫలితం **0%** నుంచి **100%** కి

1 MB తొలగిస్తే 0% – ఎందుకంటే 1 MB అనేది 64 KB కి **సరిగ్గా గుణిజం**. అన్ని సరిహద్దులూ యథాతథంగా ఉంటాయి.

1 MB + 1 byte తొలగిస్తే 100% – ఎందుకంటే ప్రతి సరిహద్దు ఒక byte జరిగింది. ముక్క 0 ఇప్పుడు పాత ముక్క 0 యొక్క bytes 1-65536 ని కలిగి ఉంది. **ఒక్క ముక్క కూడా సరిపోలలేదు**.

అంటే \$5 యొక్క "0%" ఒక విజయం కాదు – అది **అద్భుతం**. మీ సవరణ ముక్క పరిమాణానికి గుణిజం అయితే మాత్రమే.

మరియు మొదటి వరుస – చేర్చడం – ఎప్పుడూ విఫలమవుతుంది

ఒక byte చేర్చడం అంటే **ప్రతి** తర్వాతి byte ఒక స్థానం జరగడం. స్థిరమైన సరిహద్దులు స్థానం ప్రకారం పడతాయి కాబట్టి **ప్రతి ముక్కా** మారుతుంది.

16 MB • 256 ముక్కలు • **256 ముక్కలూ కొత్తవి**.

మరియు ఇది అరుదైన సందర్భం కాదు: ఒక log file మొదట్లో header రాయడం, ఒక video కి metadata చేర్చడం, ఒక document మొదట్లో ఒక పంక్తి జోడించడం – **అన్నీ ఇదే**.

7. Step – విషయాధారిత విభజన (content-defined chunking)

తప్పు ఎక్కడ జరిగిందో ఇప్పుడు స్పష్టం:

సరిహద్దుని స్థానం నిర్ణయిస్తే, స్థానం మారినప్పుడు సరిహద్దు మారుతుంది

కాబట్టి సరిహద్దుని **విషయం** నిర్ణయించాలి.

File మీద ఒక చిన్న కిటికీ (48 bytes) జరుపుతూ, ఆ కిటికీ యొక్క hash ని లెక్కించడం. ఆ hash యొక్క కింది bits సున్నా అయితే – **అక్కడ కత్తిరించు**.

ఇప్పుడు సరిహద్దు ఆ 48 bytes మీద ఆధారపడుతుంది, file lo వాటి స్థానం మీద కాదు. మొదట్లో ఒక byte చేర్చినా ఆ 48 bytes అలాగే ఉంటాయి, కాబట్టి **సరిహద్దు అక్కడే ఉంటుంది**.

```
const WIN = 48;
function chunk(buf, avg = 65536) {
  const mask = avg - 1, min = avg >> 2, max = avg << 2;
  const out = []; let start = 0, h = 0;
  for (let i = 0; i < buf.length; i++) {
    h = (Math.imul(h, 31) + buf[i]) >>> 0;
    if (i >= WIN) h = (h - Math.imul(buf[i-WIN], POW)) >>> 0; // కిటికీ నుంచి బయటికి
    const len = i - start + 1;
    if ((len >= min && (h & mask) === 0) || len >= max) {
      out.push(buf.subarray(start, i+1)); start = i+1;
    }
  }
  if (start < buf.length) out.push(buf.subarray(start));
  return out;
}
```

ఆ **min** మరియు **max** తప్పనిసరి. **min** లేకపోతే ఒక byte పొడవు ముక్కలు వస్తాయి (§13 lo mutation testing దాన్ని పట్టుకుంది); **max** లేకపోతే ఒక క్రమబద్ధమైన file కి సరిహద్దులే దొరకవు.

నేరుగా కొలిచాను:

యాదృచ్ఛిక data: ముక్కలు 45 సగటు 91.0 KB కనిష్ఠం 17970 గరిష్ఠం 245582
1 byte చేర్చాక: ముక్కలు 45 • ఒకేలా ఉన్నవి 44 (97.8%)
క్రమబద్ధమైన data: ముక్కలు 16 సగటు 256.0 KB ← సరిహద్దులు దొరకవు, max కే ఆగుతుంది

రెండు నిజాయితీ గమనికలు

1. లక్ష్యం 64 KB, వచ్చినది సగటున **91 KB**. కారణం **min** – కనిష్ఠ పరిమాణం లోపల వచ్చే సరిహద్దులు వదిలేయబడతాయి, కాబట్టి సగటు లక్ష్యం కంటే పెద్దదవుతుంది. ఇది ఒక bug కాదు, ఒక పరిణామం.

2. క్రమబద్ధమైన data మీద (ఒక నమూనా పదే పదే వచ్చే file) **సరిహద్దులే దొరకవు** – hash కూడా క్రమబద్ధంగా ఉంటుంది. అప్పుడు అది **max** దగ్గర కత్తిరిస్తుంది, అంటే స్థిరమైన ముక్కల లాగే ప్రవర్తిస్తుంది.

విషయాధారిత విభజన ఒక హామీ కాదు – అది ఒక **గణాంక** ఆస్తి.

మరియు పూర్తి పట్టిక:

సవరణ ~64 KB	మొత్తం file స్థిరమైన 4 MB స్థిరమైన 64 KB విషయాధారిత
చివర్లో 1 MB కలపడం 1.02 MB (6%)	17.00 MB (100%) 1.00 MB (6%) 1.00 MB (6%)
మధ్యలో 1 MB మార్పడం 1.04 MB (7%)	16.00 MB (100%) 4.00 MB (25%) 1.00 MB (6%)
మొదట్లో 1 byte చేర్చడం 0.08 MB (1%)	16.00 MB (100%) 16.00 MB (100%) 16.00 MB (100%)
మొదట్లో 1 MB తొలగించడం 0.02 MB (0%)	15.00 MB (100%) 15.00 MB (100%) 0.00 MB (0%)
మొదట్లో 1 MB + 1 తొలగించడం 0.02 MB (0%)	15.00 MB (100%) 15.00 MB (100%) 15.00 MB (100%)
1 byte మార్పడం 0.05 MB (0%)	16.00 MB (100%) 4.00 MB (25%) 0.06 MB (0%)

చివరి నిలుపు వరుస ఏ వరుసలోనూ 7% చాటలేదు. మరియు అది ఏ ఒక్క సవరణ రకానికీ ప్రత్యేకంగా తయారు చేయబడలేదు.

Part 3 – రెండో విరుపు: వృక్షం

8. Step – file system ఒక వృక్షం, ఒక జాబితా కాదు

ఇప్పటిదాకా files ని స్వతంత్రంగా చూశాం. అవి కాదు – అవి ఒక వృక్షంలో ఉన్నాయి, మరియు ఆ వృక్షం కూడా మారుతుంది.

ముఖ్యమైన విషయం: ఒక folder ని తరలించడం delete + create కాదు.

```
root/పని/నివేదిక/ (500 files)
    ↓ "నివేదిక" ని "పాతవి" లోపలికి లాగడం
root/పాతవి/నివేదిక/ (అవే 500 files)
```

దీన్ని delete + create గా పంపితే – 500 files delete, 500 files upload. **0 bytes మారినా**. కాబట్టి తరలింపు ఒక ప్రత్యేక ఆపరేషన్ కావాలి: `move(node, newParent)`.

ఇప్పుడు రెండు పరికరాలు offline lo ఒక్కొక్క తరలింపు చేస్తాయి. ప్రతి ఆపరేషనూ విడిగా పూర్తిగా చట్టబద్ధమైనది.

9. రెండో విరుపు – రెండు చట్టబద్ధమైన తరలింపులు, 213 files అదృశ్యం

```
మొదట: root/A/ మరియు root/B/ (B lo 50 files)
పరికరం 1: A ని B లోపలికి తరలించు → root/B/A/
పరికరం 2: B ని A లోపలికి తరలించు → root/A/B/
```

రెండూ కలిపితే: A యొక్క parent = B, B యొక్క parent = A
→ A మరియు B ఒకదాన్ని ఒకటి పట్టుకుని root నుంచి తెగిపోతాయి.
→ వాటిలోని అన్ని files *అదృశ్యం*. ఏ ఆపరేషనూ అక్రమం కాదు.

ఇది ఎంత తరచుగా జరుగుతుంది? 25 directories + 200 files ఉన్న వృక్షాలు, 20,000 యాదృచ్ఛిక జంటలు:

ప్రయోగాలు	: 20,000
వృక్షం విరిగినవి (cycle/అనాథలు)	: 243 (1.22%)
అదృశ్యమైన nodes (మొత్తం)	: 21,533
ఒక్క సంఘటనలో గరిష్టంగా పోయినవి	: 213 / 221
ఇద్దరూ *అదే* node ని తరలించినవి	: 1,068 (5.34%)

1.22% అరుదు · కాన్ జరిగినప్పుడు సగటున 88 NODES, గరిష్ఠంగా 213 / 221

ఇదే ఈ విరుపు యొక్క నిజమైన ఆకారం: తక్కువ సంభావ్యత, భారీ పరిణామం.

1.22% అంటే "వందకి ఒకసారి". ఒక కోటి వాడుకరులు రోజుకి ఒక తరలింపు చేస్తే అది రోజుకి లక్ష విరిగిన వృక్షాలు.

మరియు వాడుకరికి కనిపించేది "ఒక conflict" కాదు – నా folder మొత్తం మాయమైంది.

ఎందుకు ఇది HLD 12 యొక్క CONFLICT కంటే కష్టం

HLD 12 (Google Docs) lo రెండు ఏకకాల సవరణలని కలపవచ్చు – రెండు అక్షరాలూ ఉండగలవు.

ఇక్కడ అది సాధ్యం కాదు. "A, B లోపల ఉంది" మరియు "B, A లోపల ఉంది" – ఈ రెండూ ఏకకాలంలో నిజం కాలేవు. ఒక వృక్షం యొక్క నిర్వచనమే అది.

కాబట్టి ఇక్కడ merge ఉండదు. ఒక దాన్ని తిరస్కరించాల్సిందే, మరియు అందరూ అదే దాన్ని తిరస్కరించాలి.

10. Step – స్థితి కాదు, ఆపరేషన్లు · మొత్తం క్రమం · వలయ తిరస్కరణ

మూడు నిర్ణయాలు, మరియు మూడూ కలిపి మాత్రమే పనిచేస్తాయి.

1. ఆపరేషన్లు పంపడం, స్థితి కాదు. `move(node, dest)` – "నా వృక్షం ఇలా ఉంది" కాదు. (HLD 12 §5 యొక్క అదే పాఠం.)

2. ఒక మొత్తం క్రమం. ప్రతి ఆపరేషన్ కి `(Lamport timestamp, device id)`. ప్రతి పరికరం అదే క్రమంలో పేర్పుతుంది, కాబట్టి అందరూ అదే ఫలితానికి చేరతారు.

3. వలయం సృష్టించే తరలింపుని వదిలేయడం.

```
apply(ops) {
  const sorted = ops.slice().sort((a,b) =>
    a.ts - b.ts || (a.dev < b.dev ? -1 : a.dev > b.dev ? 1 : 0));
  for (const op of sorted) {
    if (op.type === 'move') {
      if (this.#wouldCycle(op.node, op.dest)) { skipped++; continue; } // ← §9
      this.#parent.set(op.node, op.dest);
    }
    ...
  }
}
```

"వదిలేయడం" ఏకపక్షంగా అనిపిస్తుంది - కానీ అది ఏకపక్షం కాదు

క్రమం మొత్తమైనది కాబట్టి, ప్రతి పరికరం అదే ఆపరేషన్ ని వదిలేస్తుంది. వాడుకరి 1 యొక్క తరలింపు నిలుస్తుంది, వాడుకరి 2 యొక్క వదిలేయబడుతుంది - అందరి దగ్గరూ.

ఇది "సరైన" జవాబు కాదు (సరైన జవాబు లేదు). ఇది ఒక స్థిరమైన జవాబు, మరియు ఒక వ్యక్తానికి అదే అవసరం.

వాడుకరికి చెప్పాల్సింది: "మీ తరలింపు వర్తింపజేయలేకపోయాం" - మౌనంగా వదిలేయకుండా.

ఒక సూక్ష్మమైన షరతు, మరియు FUZZ దాన్ని పట్టుకుంది

ఆ క్రమం నిజంగా మొత్తమైనది కావాలంటే (ts, dev) జంట ప్రత్యేకంగా ఉండాలి.

నా మొదటి harness ఒక పరికరానికి ఒకే timestamp తో రెండు ఆపరేషన్లు ఇచ్చింది. అప్పుడు comparator నున్నా ఇస్తుంది, JavaScript యొక్క stable sort input క్రమాన్ని ఉంచుతుంది - మరియు రెండు పరికరాలు వేర్వేరు వ్యక్తాలకి చేరతాయి. 200 lo 11 విఫలం.

పరిష్కారం code lo కాదు - ప్రతి పరికరం తన Lamport గడియారాన్ని ప్రతి ఆపరేషన్ కీ పెంచాలి. ఆ షరతుని ఇప్పుడు code lo ఒక వ్యాఖ్యగా రాశాను, ఎందుకంటే అది పాటించకపోతే మౌనంగా విరుగుతుంది.

మరియు delete కూడా fuzz వల్లే సరిదిద్దబడింది: ఒక directory ని తొలగించినప్పుడు దాని కింది మొత్తం subtree పోవాలి. లేకపోతే దాని పిల్లలు అనాథలవుతారు - 200/200 విఫలం.

Part 4 – మూడో విరుపు: ఎన్నిసార్లు అడగాలి

11. మూడో విరుపు – 13.9 గంటలు నిరీక్షణ, 5.5 నిమిషాలు పని

ఇప్పుడు §1 యొక్క రెండో సగం – చిన్న **90% files**. 10 లక్షల files, సగటు 4 KB, 100 Mbps, round-trip 50 ms:

1 మిలియన్ files • సగటు 4 KB • మొత్తం 4.1 GB
network: 100 Mbps • round-trip 50 ms

కేవలం bytes పంపడానికి పట్టే సమయం : 5.5 ని
కేవలం round-trips కి పట్టే సమయం : 13.9 గం ← ఒక్కో file కి ఒక request

13.9 గంటలు VS 5.5 నిమిషాలు – 152 రెట్లు

ఆ 13.9 గంటల్లో మీ network దాదాపు పూర్తిగా ఖాళీగా ఉంటుంది. మీరు bytes పంపడం లేదు – మీరు **server జవాబు కోసం వేచి ఉన్నారు**, 10 లక్షల సార్లు.

మరియు ఇది "నెమ్మదైన network" సమస్య కాదు. Network ని 10 రెట్లు వేగవంతం చేస్తే 5.5 నిమిషాలు 33 సెకన్లు అవుతాయి, మరియు **13.9 గంటలు అలాగే ఉంటాయి**.

12. Step – ముద్ద చేయడం • మరియు సమతుల్య file పరిమాణం

ఎలా పంపుతున్నాం	round-trip సమయం	మొత్తం సమయం
ఒక్కొక్కటి, వరుసగా	13.9 గం	13.9 గం
ఒక్కొక్కటి, 16 ఏకకాలం	52.1 ని	52.1 ని
ఒక్కొక్కటి, 64 ఏకకాలం	13.0 ని	13.0 ని
64 ఏకకాలం, 100 files ముద్ద	7.8 సె	5.5 ని
64 ఏకకాలం, 1000 files ముద్ద	0.8 సె	5.5 ని

ఏకకాలత సహాయపడుతుంది; ముద్ద చేయడం **PROBLEM** ని తొలగిస్తుంది

64 ఏకకాలం round-trip సమయాన్ని 13.9 గంటల నుంచి 13 నిమిషాలకి తగ్గిస్తుంది – 64 రెట్లు. బాగుంది, మరియు అది ఇప్పటికీ **bandwidth సమయం కంటే రెండింతలు**.

1000-file ముద్దలతో round-trip సమయం **0.8 సెకన్లు**. ఇప్పుడు మొత్తం సమయం 5.5 నిమిషాలు – అంటే **bytes పంపడానికి పట్టే సమయం**.

అదే లక్ష్యం: మీ **system network** యొక్క వేగంతో పరిమితం కావాలి, దాని ఆలస్యంతో కాదు.

ఆ పరిమితి ఎక్కడ మారుతుందో ఒక సూత్రం ఉంది – **round-trip × bandwidth**:

round-trip	network వేగం	సమతుల్య file పరిమాణం
5 ms	100 Mbps	61 KB
5 ms	1000 Mbps	610 KB
50 ms	100 Mbps	610 KB
50 ms	1000 Mbps	6.0 MB
200 ms	100 Mbps	2.4 MB
200 ms	1000 Mbps	23.8 MB

వేగవంతమైన **NETWORK** ఈ **PROBLEM** ని పట్టది చేస్తుంది

100 Mbps మీద సమతుల్య పరిమాణం 610 KB. 1 Gbps మీద అది **6 MB**.

అంటే network వేగం పెరిగిన కొద్దీ **ఇంకా ఎక్కువ files** round-trip-పరిమితంగా మారతాయి. §1 ప్రకారం p99 file 2.5 MB – కాబట్టి 1 Gbps మీద మీ files lo **99% కంటే ఎక్కువ** ఆలస్యంతో పరిమితం.

Bandwidth కొనవచ్చు. Round-trip కొనలేరు – అది కాంతి వేగం మరియు భూగోళం.

Part 5 – నిరూపణ మరియు ప్రదర్శన

13. మొత్తం code • 340 MB • mutation testing

ఏడు నియమాలు: ముక్కలు కలిపితే అసలు file, పంపిన bytes ≤ file పరిమాణం, ముక్క పొడవులు min/max lo, మారని file → 0 bytes, bytes జరిపినా సగానికి పైగా ముక్కలు బతకాలి, ప్రతి node root నుంచి చేరాలి, ఆపరేషన్ల క్రమం మారినా అదే వృక్షం.

200 యాదృచ్ఛిక runs • 600 files • 340 MB • 4,906 ముక్కలు
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

files	600
chunks	4,906
bytes	35,69,67,663
sent	35,69,67,663
moves	5,662
creates	1,544
deletes	794
skipped	2,770
resync	600

ఆ skipped 2,770 ముఖ్యమైనది: 7,206 ఆపరేషన్లలో **2,770 తిరస్కరించబడ్డాయి** (వలయం, తొలగించిన node, లేదా లేని parent). అంటే \$10 యొక్క రక్షణలు నిజంగా పరీక్షించబడ్డాయి.

==== mutation testing ====

మార్పు లేని code	→ 0/200 విఫలం
వలయ తనిఖీ తీసేస్తే	→ 55/80 విఫలం
ఉదా: 1 nodes root నుంచి చేరలేవు	
విషయాధారిత విభజనకి బదులు స్థిరమైన 64 KB	→ 80/80 విఫలం
ఉదా: file 0: bytes జరిపాక 0/11 ముక్కలే బతికాయి	
కనిష్ట ముక్క పరిమాణం తీసేస్తే	→ 80/80 విఫలం
ఉదా: file 0: ముక్క 0 పొడవు 1	
ఆపరేషన్ల మొత్తం క్రమంలో పేర్లకపోతే	→ 76/80 విఫలం
ఉదా: క్రమం మారితే వేరే వృక్షం వచ్చింది	
తొలగించిన nodes కి కూడా తరలింపు అనుమతిస్తే	→ 63/80 విఫలం
ఉదా: 9 nodes root నుంచి చేరలేవు	
ముక్క id ని విషయం నుంచి కాకుండా స్థానం నుంచి తీస్తే	→ 80/80 విఫలం
ఉదా: file 0: మారని file కి 616383 bytes పంపాం	

ఆరు మార్పులు, ఆరు పట్టుబడ్డాయి - ఈ SERIES LO మొదటిసారి

మరియు రెండో వరుస ముఖ్యమైనది: "విషయాధారిత విభజనకి బదులు స్థిరమైన 64 KB" - ఇది ఒక bug కాదు, ఇది **§5 యొక్క design**.

దాన్ని ఒక mutation గా పెట్టడం అంటే, §6 lo నేను ఒక పట్టికలో చూపించిన విషయాన్ని ఇప్పుడు **ఒక నియమం** గా నిరూపించడం: 0/11 ముక్కలే బతికాయి .

ఒక design నిర్ణయాన్ని mutation గా పరీక్షించగలిగితే, అది ఒక అభిప్రాయం కాదు.

Fuzz రాసే దారిలో అది **నా design lo రెండు నిజమైన తప్పులు** పట్టుకుంది:

1. DIRECTORY ని తొలగిస్తే దాని పిల్లలు అనాథలవుతారు - 200/200 విఫలం

నా మొదటి delete ఒక్క node ని మాత్రమే చనిపోయినట్టు గుర్తించింది. దాని పిల్లలు బతికే ఉన్నారు, కానీ వాళ్ళ parent చనిపోయాడు - కాబట్టి వాళ్ళు root నుంచి **చేరలేరు**.

సరైన semantics: directory తొలగిస్తే **subtree మొత్తం** పోతుంది. ఇది స్పష్టంగా అనిపిస్తుంది - మరియు నేను దాన్ని రాయలేదు.

2. "మొత్తం క్రమం" నిజంగా మొత్తం కాదు - 200 LO 11 విఫలం

§10 lo వివరించాను: (ts, dev) జంట ప్రత్యేకంగా లేకపోతే, stable sort input క్రమాన్ని ఉంచుతుంది, మరియు రెండు పరికరాలు వేర్వేరు వృక్షాలకి చేరతాయి.

ఇది **code యొక్క bug కాదు, code యొక్క ఒక చెప్పని షరతు** - మరియు fuzz అదే వెలికితీసింది.

మరియు నా **నియమంలో** ఒక తప్పు కూడా: "bytes జరిపాక సగానికి పైగా ముక్కలు బతకాలి" అనే నియమం, file మొత్తం **ఒకే ముక్క** అయినప్పుడు అర్థరహితం (ఆ ఒక్క ముక్క మారక తప్పదు). నాలుగు ముక్కల కంటే తక్కువ ఉన్న files ని ఆ నియమం నుంచి మినహాయించాక - 0/200.

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి	ఏ సంఖ్య చెప్పాలి
0-6	Requirements · ఇది రెండు problems	చిన్న 90% = bytes lo 13% , ops lo 90%
6-12	పూర్తి file upload → ముక్కలు	1 byte → 16 MB ; 64 KB ముక్కలు → 0%
12-20	మొదట్లో ఒక byte → స్థిరమైన ముక్కలు విఫలం	0% → 100% , ఒక్క byte తేడాతో
20-26	విషయాధారిత విభజన	ఏ సవరణకీ 7% దాటలేదు · 97.8% ముక్కలు బతికాయి
26-34	వృక్షం - రెండు చట్టబద్ధమైన తరలింపులు	1.22% , కానీ 213/221 nodes

నిమిషాలు	ఏమి చెప్పాలి	ఏ సంఖ్య చెప్పాలి
34-38	ఆపరేషన్లు + మొత్తం క్రమం + వలయ తిరస్కరణ	"ఒకే జవాబు, అందరి దగ్గర"
38-45	Round-trips → ముద్ద	13.9 గో vs 5.5 ని · సమతుల్య పరిమాణం 610 KB

ఈ PROBLEM LO మిమ్మల్ని వేరుగా నిలబెట్టే ఒక్క క్షణం

చాలామంది "file ని ముక్కలుగా విడగొట్టి మారిన ముక్కలని పంపుతాను" అని చెప్పి ముందుకి వెళ్తారు. అది సరైనది, మరియు అది **సగం జవాబు**.

మీరు ఆగి ఇలా అడగండి: **"ఒక 2 GB file మొదట్లో ఒక byte చేర్చితే? స్థిరమైన ముక్కలతో ప్రతి సరిహద్దు ఒక byte జరుగుతుంది, కాబట్టి ఒక్క ముక్క కూడా సరిపోలదు – నేను కొలిచినప్పుడు అది 100%, అంటే మొత్తం 2 GB. అందుకే సరిహద్దుని స్థానం కాదు, విషయం నిర్ణయించాలి. Rolling hash తో అది 1% కి పడింది."**

ఆ ఒక్క ప్రశ్న అడగడం – మరియు దానికి మీ సొంత సంఖ్య కలిగి ఉండటం – మీరు దీన్ని **చదవలేదు, నడిపారు** అని చూపుతుంది.

15. నోటీతో చెప్పాల్సిన English script

Scope గురించి:

"I'd start by separating two problems that a single average hides. If I model a realistic file-size distribution, the largest one percent of files hold more than half the bytes, and the smallest ninety percent hold thirteen percent of the bytes but ninety percent of the operations. Those need completely different fixes – delta encoding for the big ones, batching for the small ones – and doing only one still leaves the sync slow."

Chunking గురించి:

"Uploading whole files means a one-byte edit to a sixteen-megabyte file costs sixteen megabytes. Fixed-size chunks help a lot for in-place edits – sixty-four kilobyte chunks took a one-byte change down to sixty-four kilobytes. But I want to flag where that breaks, because it's not an edge case. If you insert one byte at the front, every boundary shifts by one, so no chunk matches and you re-upload a hundred percent. I measured something even sharper: deleting exactly one megabyte re-uploads zero percent, and deleting one megabyte *plus one byte* re-uploads a hundred percent. The zero was alignment luck. So boundaries have to be decided by content, not position – a rolling hash over a forty-eight-byte window, cut where the low bits are zero. That kept every edit type under seven percent."

The tree గురించి:

"Moves are the hard part, and not for the reason people expect. Moving a folder isn't delete-plus-create – that would re-upload everything for zero changed bytes. So move is its own operation, and now two devices can make two individually legal moves that are jointly impossible: A into B, and B into A. I ran twenty thousand random pairs: one point two percent broke the tree, and when they broke it, an average of eighty-eight nodes became unreachable – up to two hundred thirteen out of two hundred twenty-one. Unlike a text edit there's no merge here; a tree can't satisfy both. So you need a total order over operations and a rule that skips a move that would create a cycle – every device skips the *same* one, and you tell the user their move didn't apply rather than losing it silently."

Round trips గురించి:

"Last, a million small files. The bytes take five and a half minutes. The round trips take thirteen point nine hours – one request per file at fifty milliseconds. Concurrency helps: sixty-four in flight brings it to thirteen minutes. Batching removes it: a thousand files per request brings round-trip time to under a second, and the sync becomes bandwidth-bound at five and a half minutes. And the crossover is round-trip times bandwidth – six hundred ten kilobytes at fifty milliseconds and a hundred megabits. Note that a faster network makes this worse: at a gigabit the crossover is six megabytes, so an even larger share of files is latency-bound. You can buy bandwidth; you can't buy round trips."

16. Follow-ups – dedup, cache, LAN sync, పెద్ద folders

"వేర్వేరు వాడుకరుల దగ్గర ఒకే file ఉంటే?"

విషయాధారిత విభజన ఇక్కడ ఉచితంగా ఒక లాభం ఇస్తుంది: ముక్క id దాని **విషయం** నుంచి వస్తుంది, కాబట్టి ఒకే ముక్క ఎవరి దగ్గర ఉన్నా అదే id. Server దగ్గర ఆ ముక్క ఇప్పటికే ఉంటే – **upload అసలు జరగదు**. ఒక ప్రసిద్ధ PDF ని లక్షమంది పంచుకుంటే server దాన్ని ఒకసారి ఉంచుతుంది. కానీ ఒక భద్రతా ప్రశ్న ఉంది: "ఈ file ఇప్పటికే ఉంది" అనే జవాబు **ఒక సమాచార లీక్** – ఒక దాడి చేసేవాడు ఒక file ఉందో లేదో పరీక్షించగలడు. అందుకే నిజమైన systems ఈ dedup ని ఒక వాడుకరి లోపలే పరిమితం చేయడం లేదా upload ని పూర్తి చేయించడం చేస్తాయి.

"ఏమి మారినట్టే ఎలా తెలుస్తుంది?"

రెండు దారులు, మరియు రెండూ కావాలి. **File system watcher** (Windows lo

ReadDirectoryChangesW, macOS lo FSEvents, Linux lo inotify) – తక్షణమే, కానీ **నమ్మదగినది**

కాదు: buffer నిండితే సంఘటనలు పోతాయి, మరియు client ఆగి ఉన్నప్పుడు జరిగినవి రావు. కాబట్టి

అప్పుడప్పుడూ ఒక **పూర్తి scan** కూడా కావాలి, మరియు §11 యొక్క అదే problem అక్కడ వస్తుంది – 10 లక్షల files ని scan చేయడం ఒక metadata ఆపరేషన్, ఒక bytes ఆపరేషన్ కాదు.

"ఒక LAN lo రెండు పరికరాలు ఉంటే?"

LAN sync – పరికరాలు ఒకదానికొకటి నేరుగా ముక్కలు పంపుకోవడం. ఇది §11 కి **సహాయపడుతుంది కూడా**: LAN round-trip 50 ms కాదు, 1 ms. కాబట్టి సమతుల్య పరిమాణం 610 KB నుంచి **12 KB** కి పడుతుంది. Metadata మాత్రం server ద్వారానే వెళ్ళాలి – లేకపోతే §10 యొక్క మొత్తం క్రమం ఎవరు నిర్ణయిస్తారు?

"ఒక folder lo 10 లక్షల files ఉంటే?"

ఇక్కడ మీ metadata నిల్వ ఒక వృక్షంగా ఉండటం సమస్య అవుతుంది: ఒక directory ని తరలీనిస్తే దాని కింది 10 లక్షల paths మారతాయి. **పరిష్కారం: path ని నిల్వ చేయకపోవడం** – ప్రతి node దగ్గర కేవలం (id, parentId, name). అప్పుడు తరలింపు **ఒక్క row update**. Path కావాలంటే అది **లెక్కించబడుతుంది**, నిల్వ చేయబడదు. (ఇదే నమూనా ఈ series lo చాలాసార్లు వచ్చింది: *derived, not stored.*)

"ఈ doc lo నేను ఏమి కొలవలేదు"

రెండు విషయాలు. **Delta encoding ముక్కలోపల** – నేను ఒక ముక్క మారితే మొత్తం ముక్కని పంపుతాను; నిజమైన systems ఆ ముక్క యొక్క binary diff ని పంపగలవు. రెండేది – **ఆపరేషన్లని అంతకుముందు వర్తింపజేసిన వాటితో కలిపి తిరిగి లెక్కించడం**. నా apply ప్రతిసారీ మొత్తం క్రమాన్ని మళ్ళీ పేర్చుతుంది; నిజమైన systems ఒక కొత్త ఆపరేషన్ మధ్యలో చేరినప్పుడు **వెనక్కి తిప్పి మళ్ళీ వర్తింపజేయాలి** (Kleppmann యొక్క move CRDT). నేను దాన్ని అమలు చేయలేదు, కాబట్టి దాని ఖర్చు గురించి నా దగ్గర సంఖ్య లేదు.

17. ఏమి నేర్చుకున్నాం

1. ఒక సగటు సంఖ్య రెండు problems ని దాచగలదు. "సగటు file 174 KB" అనేది నిజం, మరియు అది **median కంటే 18 రెట్లు**. చిన్న 90% files = ops lo 90%, bytes lo 13%.

2. ముక్కలు చేయడం సగం జవాబు; సరిహద్దులు ఎక్కడ పడతాయో అనేది మిగతా సగం. స్థిరమైన ముక్కలతో మొదట్లో ఒక byte చేర్చడం = **100%**.

3. "0%" ఒక విజయం కాదు – అది అదృష్టం కావచ్చు. 1 MB తొలగిస్తే 0%; 1 MB + 1 byte తొలగిస్తే **100%**. తేడా ఒక్క byte.

4. వేగవంతమైన network కొన్ని problems ని పెద్దవి చేస్తుంది. సమతుల్య పరిమాణం 100 Mbps మీద 610 KB, 1 Gbps మీద **6 MB** – అంటే ఇంకా ఎక్కువ files ఆలస్యంతో పరిమితం.

5. Bandwidth కొనవచ్చు; round-trip కొనలేరు. 13.9 గంటల్లో మీ network ఖాళీగా ఉంది.

6. అన్ని conflicts ని కలపలేము. HLD 12 lo రెండు అక్షరాలూ ఉండగలవు. ఇక్కడ "A, B lo" మరియు "B, A lo" **ఏకకాలంలో నిజం కాలేవు** – కాబట్టి ఒకదాన్ని తిరస్కరించాలి, మరియు అందరూ **అదే** దాన్ని.

7. అరుదైనది అంటే చిన్నది కాదు. 1.22% మాత్రమే విరుగుతాయి, మరియు విరిగినప్పుడు **221 lo 213** files అదృశ్యమవుతాయి.

8. ఒక "చెప్పని షరతు" ఒక bug లాగే విరుస్తుంది. నా మొత్తం క్రమానికి (ts, dev) ప్రత్యేకత అవసరం, మరియు నేను దాన్ని ఎక్కడా రాయలేదు – 200 lo 11 విఫలం.

9. ఒక design నిర్ణయాన్ని కూడా mutation గా పరిక్షించవచ్చు. "స్థిరమైన ముక్కలు వాడు" అనేది ఒక bug కాదు, ఒక ఎంపిక – మరియు అది 80/80 విఫలమవుతుంది.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

File sync అంటే files పంపడం కాదు – అది "ఏమి మారింది" అనే ప్రశ్నకి జవాబు చెప్పడం, మరియు ఆ ప్రశ్నకి మూడు వేర్వేరు అర్థాలు ఉన్నాయి.

ఒక file లోపల ఏమి మారింది → విషయాధారిత విభజన (§7).

వ్యక్తం lo ఏమి మారింది → ఆపరేషన్లు మరియు ఒక మొత్తం క్రమం (§10).

ఎన్ని విషయాలు మారాయి → ముద్ద చేయడం (§12).

మూడూ "తేడా" గురించే, మరియు మూడింటికీ వేర్వేరు పరిష్కారాలు. ఒకదాన్ని రెండోదానితో పరిష్కరించలేరు.